JAVA SCRIPT

CHAPTER-1

1.1 INTRODUCTION

One of many JavaScript HTML methods is getElementById (). The example below "finds" an HTML element (with id="demo"), and changes the element content (innerHTML) to "Hello JavaScript":

PROGRAM 1:

```
<!DOCTYPE html>
<html>
<body>
<h2>What Can JavaScript Do?</h2>
<p id="demo">JavaScript can change HTML content.</p>
<button type="button" onclick='document.getElementById("demo").innerHTML =
"Hello JavaScript!"'>Click Me!</button>
</body>
</html>
```

1.1.1 Changing HTML Attribute Values using JavaScript

In this example JavaScript changes the value of the src (source) attribute of an <img> tag:

PROGRAM 2:

```
<!DOCTYPE html>
<Html>
<Body>
<h2>What Can JavaScript Do? </h2>
<p>JavaScript can change HTML attribute values. </p>
<p>In this case JavaScript changes the value of the src (source) attribute of an image.
</p>
<button onclick="document.getElementById ('my Image').src='pic_bulbon.gif'">Turn
on the light</button>
<img id="my Image" src="pic_bulboff.gif" style="width: 100px">
<button onclick="document.getElementById ('my Image').src='pic_bulboff.gif'">Turn
off the light</button>
</body>
</html>
```

1.1.2 Changing Html Styles using JavaScript

Changing the style of an HTML element, is a variant of changing an HTML attribute:

PROGRAM 3:

```
<!DOCTYPE html>
<html>
<body>
<h2>What Can JavaScript Do?</h2>
<p id="demo">JavaScript can change the style of an HTML element.</p>
<button type="button"
onclick="document.getElementById('demo').style.fontSize='35px'">Click
Me!</button>
</body>
</html>
```

1.1.3 The <Script> Tag

In HTML, JavaScript code is inserted between <script> and </script> tags.

PROGRAM 4:

```
<!DOCTYPE html>
<Html>
<Body>
<h2>JavaScript in Body</h2>
<p id="demo"></p>
<Script>
document.getElementById ("demo").innerHTML = "My First JavaScript";
</script>
</body>
</html>
```

1.2 FUNCTIONS

A JavaScript function is a block of JavaScript code that can be executed when "called" for. For example, a function can be called when an **event** occurs, like when the user clicks a button.

A JavaScript function is placed in the <head> section of an HTML page.

PROGRAM 5:

```
<!DOCTYPE html>
<Html>
<Head>
<script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>
</head>
<body>
<h2>Demo JavaScript in Head</h2>
<p id="demo">A Paragraph.</p>
<button type="button" onclick="myFunction()">Try it</button>
</body>
</html>
```

A JavaScript function is also placed in the <body> section of an HTML page.

PROGRAM 6:

```
<!DOCTYPE html>
<html>
<body>
<h2>Demo JavaScript in Body</h2>
<p id="demo">A Paragraph.</p>
<button type="button" onclick="myFunction()">Try it</button>
<script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>

</body>
</html>
```

1.3 EXTERNAL JAVASCRIPT

Scripts can also be placed in external files, External scripts are practical when the same code is used in many different web pages. JavaScript files have the file extension **.js**. To use an external script, put the name of the script file in the src (source) attribute of a <script> tag:

```
Function myFunction () {
  document.getElementById ("demo").innerHTML = "Paragraph changed”;
}
```

Save as **myScript.js**

PROGRAM 7:

```
<!DOCTYPE html>
<html>
<body>
<h2>Demo External JavaScript</h2>
<p id="demo">A Paragraph.</p>
<button type="button" onclick="myFunction()">Try it</button>
<p>This example links to "myScript.js".</p>
<p>(myFunction is stored in "myScript.js")</p>
<script src="myScript.js"></script>
</body>
</html>
```

1.4 JAVASCRIPT DISPLAY

JavaScript can "display" data in different ways:

- innerHTML
- Document. Write()
- Window. Alert()
- console.log()

1.3.1 InnerHTML

JavaScript can use the document.getElementById (id) method. The id attribute defines the HTML element. The innerHTML property defines the HTML content:

PROGRAM 8:

```
<!DOCTYPE html>
<html>
<body>
<h2>My First Web Page</h2>
```

```
<p>My First Paragraph.</p>
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML = “5 + 6”;
</script>
</body>
</html>
```

1.4.2 Document. Write ()

PROGRAM 9:

```
<!DOCTYPE html>
<Html>
<Body>
<h2>My First Web Page</h2>
<p>my first paragraph. </p>
<p>Never call document. Write after the document has finished loading. It will
overwrite the whole document. </p>
<Script>document. Write (5 + 6);
</script>
</body>
</html>
```

1.4.3 Window. Alert ()

In JavaScript, The Window Object Is The Global Scope Object. This Means That Variables, Properties, And Methods By Default Belong To The Window Object. This Also Means That Specifying The **Window** Keyword Is Optional.

PROGRAM 10

```
<!DOCTYPE html>
<Html>
<Body>
<h2>My First Web Page</h2>
<p>my first paragraph. </p>
<Script>
Window. Alert (5 + 6);
</script>
</body>
</html>
```

1.4.4 Console. Log ()

PROGRAM 11:

```
<!DOCTYPE html>
<Html>
<Body>
<h2>Activate Debugging</h2>
<p>F12 on your keyboard will activate debugging. </p>
<p>then select "Console" in the debugger menu, then click Run again. </p>
<Script>
console.log (5 + 6);
</script>
</body>
</html>
```

1.5 JAVASCRIPT STATEMENTS

JavaScript statements are Values, Operators, Expressions, Keywords, and Comments.

JavaScript Expressions

JavaScript statements can be grouped together in code blocks, inside curly brackets {...}.The purpose of code blocks is to define statements to be executed together.

PROGRAM 12:

```
<!DOCTYPE html>
<html>

<body>
<h2>JavaScript Expressions</h2>
<p>Expressions compute to values.</p>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = 5 * 10;
</script>

</body>
</html>
```

JavaScript Keywords

JavaScript statements often start with a **keyword** to identify the JavaScript action to be performed. Our Reserved Words Reference lists all JavaScript keywords.

List of Some of the Keywords

Keywor	Description
var	Declares a variable
let	Declares a block variable
const	Declares a block constant
if	Marks a block of statements to be executed on a condition
switch	Marks a block of statements to be executed in different cases
for	Marks a block of statements to be executed in a loop
function	Declares a function
return	Exits a function
try	Implements error handling to a block of statements

1.6 JAVASCRIPT VALUES

The JavaScript syntax defines two types of values:

- Fixed values
- Variable values

Fixed values are called **Literals**.
Variable values are called **Variables**.

1.6.1 JavaScript Literals

The two most important syntax rules for fixed values are:

1. **Numbers** are written with or without decimals.
2. Strings **are text, written within double or single quotes.**

PROGRAM 13:

```
<! DOCTYPE html>
<Html>
<Body>
<h2>JavaScript Strings</h2>
<p>Strings can be written with double or single quotes. </p>
<p id="demo"></p>
<Script>
document.getElementById ("demo").innerHTML = 'John Doe, 105.2’;
</script>
</body>
</html>
```

1.6.2 JavaScript Variables

Variables are used to **store** data values. JavaScript uses the keywords var, let and const to **declare** variables. An **equal sign** is used to **assign values** to variables.

PROGRAM 14:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Variables</h2>
<p>In this example, x is defined as a variable.
Then, x is assigned the value of 6:</p>
<p id="demo"></p>
<script>
let x;
x = 6;
document.getElementById("demo").innerHTML = x;
</script>
</body>
</html>
```

1.7 COMMENTS

Single Line Comments

Single line comments start with //.
Any text between // and the end of the line will be ignored by JavaScript (will not be executed).

PROGRAM 15:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Comments</h2>
<p id="demo"></p>
<script>
let x = 5;      // Declare x, give it the value of 5
let y = x + 2;  // Declare y, give it the value of x + 2
document.getElementById("demo").innerHTML = y;
</script>
</body>
</html>
```

Multi Line Comments

Multi-line comments start with /* and end with */.Any text between /* and */ will be ignored by JavaScript.

PROGRAM 16:

```
<!DOCTYPE html>
<html>
<body>
<h1 id="myH"></h1>
<p id="myP"></p>
<script>
/*
The code below will change
the heading with id = "myH"
and the paragraph with id = "myP"
*/
document.getElementById("myH").innerHTML = "JavaScript Comments";
document.getElementById("myP").innerHTML = "My first paragraph.";
</script>
</body>
</html>
```

1.8 JAVASCRIPT IDENTIFIERS

All JavaScript **variables** must be **identified** with **unique names**. These unique names are called **identifiers**. Identifiers can be short names (like x and y) or more descriptive names (age, sum, total Volume).

The general rules for constructing names for variables (unique identifiers) are:

- Names can contain letters, digits, underscores, and dollar signs.
- Names must begin with a letter.
- Names can also begin with $ and _ (but we will not use it in this tutorial).
- Names are case sensitive (y and Y are different variables).
- Reserved words (like JavaScript keywords) cannot be used as names.

1.9 CONST ARRAY AND OBJECT

The property of a const object can be changed but it cannot be changed to a reference to the new object. The values inside the const array can be changed, it can add new items to const arrays but it cannot reference a new array.

Array:

Change the elements of a constant array:

PROGRAM 17:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript const</h2>
<p>Declaring a constant array does NOT make the elements unchangeable:</p>
<p id="demo"></p>
<script>
// Create an Array:
const cars = ["Saab", "Volvo", "BMW"];
// Change an element:
cars[0] = "Toyota";
// Add an element:
cars.push("Audi");
// Display the Array:
document.getElementById("demo").innerHTML = cars;
</script>
</body>
</html>
```

Object:

You can change the properties of a constant object:

PROGRAM 18:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript const</h2>
<p>Declaring a constant object does NOT make the objects properties unchangeable:</p>
<p id="demo"></p>
<script>
// Create an object:
const car = {type:"Fiat", model:"500", color:"white"};
// Change a property:
car.color = "red";
// Add a property:
car.owner = "Johnson";
// Display the property:
document.getElementById("demo").innerHTML = "Car owner is " + car.owner;
</script>
</body>
</html>
```

CHAPTER-2

2.1 OPERATORS

In JavaScript, an operator is a special symbol used to perform operations on operands (values and variables).

Types of JavaScript Operators

There are different types of JavaScript operators:

- Arithmetic Operators
- Assignment Operators
- Comparison Operators
- String Operators
- Logical Operators
- Bitwise Operators
- Ternary Operators
- Type Operators

Arithmetic Operators

Arithmetic Operators are used to perform arithmetic on numbers:

operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

PROGRAM 1:

```
<!DOCTYPE html>
<html>
<Body>
<h1>JavaScript Arithmetic</h1>
<h2>Arithmetic Operations</h2>
<p>a typical arithmetic operation takes two numbers (or expressions) and produces
a new number. </p>
<p id="demo"></p>
<Script>
let a = 3;
let x = (100 + 50) * a;
document.getElementById ("demo").innerHTML = x;
</script>
</body>
```

```
</html>
```

Assignment Operators

Assignment operators assign values to JavaScript variables.

Operator	Example	Same As	Name
=	x = y	x = y	assignment
+=	x += y	x = x + y	add and assign
-=	x -= y	x = x - y	subract and assign
*=	x *= y	x = x * y	multiply and assign
/=	x /= y	x = x / y	divide and assign
%=	x %= y	x = x % y	mod and assign
**=	x **= y	x = x ** y	Exponentiation Assignment Operator

Shift Assignment Operators

Operator	Example	Same As
<<=	x <<= y	x = x << y
>>=	x >>= y	x = x >> y
>>>=	x >>>= y	x = x >>> y

Bitwise Assignment Operators

Operator	Example	Same As
&=	x &= y	x = x & y
^=	x ^= y	x = x ^ y

Logical Assignment Operators

Operator	Example	Same As
&&=	x &&= y	x = x && (x = y)
||=	x ||= y	x = x || (x = y)
??=	X??= y	x = x?? (x = y)

The **Addition Assignment Operator** (+=) adds a value to a variable.

PROGRAM 2:

```
<!DOCTYPE html>
<Html>
<Body>
<h1>JavaScript Arithmetic</h1>
<h2>the += Operator</h2>
<p id="demo"></p>
<Script>
var x = 10;
x += 5;
document.getElementById ("demo").innerHTML = x;
</script>
</body>
</html>
```

Comparison Operators

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

String Comparison

All the comparison operators above can also be used on strings:

PROGRAM 3:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript String Operators</h1>
<p>All conditional operators can be used on both numbers and strings.</p>
<p id="demo"></p>
<script>
let text1 = "A";
let text2 = "B";
let result = text1 < text2;
document.getElementById("demo").innerHTML = "Is A less than B? " + result;
</script>
</body>
</html>
```

Logical Operators

Operator	Description
&&	logical and
||	logical or
!	logical not

Type Operators

Operator	Description
typeof	Returns the type of a variable
instanceof	Returns true if an object is an instance of an object type

Bitwise Operators

Bit operators work on 32 bits numbers. Any numeric operand in the operation is converted into a 32 bit number. The result is converted back to a JavaScript number.

Operator	Description	Example	Same as	Result	Decimal
&	AND	5 & 1	0101 & 0(	1	1
|	OR	5 | 1	0101 | 0C	101	5
~	NOT	~ 5	~0101	1010	10
^	XOR	5 ^ 1	0101 ^ 0C	100	4
<<	left shift	5 << 1	0101 << 1	1010	10
>>	right shift	5 >> 1	0101 >> 1	10	2
>>>	unsigned right shift	5 >>> 1	0101 >>> 1	0	2

2.2 DATA TYPES

Primitive data types:

There are eight types of primitive data types in JavaScript.

- String
- Number
- BigInt
- Boolean
- Undefined
- Null
- Symbol
- Object

Non-primitive data types:
The object data type can contain:

- An object
- An array
- A date

Examples:

```
// Numbers:
let length = 16;
let weight = 7.5;
// Strings:
let color = "Yellow";
Let last Name = "Johnson";
// Booleans
let x = true;
let y = false;
// Object:
const person = {firstName:"John", lastName:"Doe"};
// Array object:
const cars = ["Saab", "Volvo", "BMW"];
// Date object:
const date = new Date ("2022-03-25");
```

Strings
A string (or a text string) is a series of characters like "John Doe".Strings are written with quotes. You can use single or double quotes:

PROGRAM 4:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Strings</h2>
<p>Strings are written with quotes. You can use single or double quotes:</p>
<p id="demo"></p>
<script>
let carName1 = "Volvo XC60";
let carName2 = 'Volvo XC60';
document.getElementById("demo").innerHTML =
carName1 + "<br>" +
carName2;
</script>
</body>
</html>
```

Numbers

All JavaScript numbers are stored as decimal numbers (floating point).Numbers can be written with, or without decimals:

PROGRAM 5:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Numbers</h2>
<p>Numbers can be written with, or without decimals:</p>
<p id="demo"></p>
<script>
let x1 = 34.00;
let x2 = 34;
let x3 = 3.14;
document.getElementById("demo").innerHTML =
x1 + "<br>" + x2 + "<br>" + x3;
</script>
</body>
</html>
```

BigInt

All JavaScript numbers are stored in a 64-bit floating-point format. JavaScript BigInt is a new data type (ES2020) that can be used to store integer values that are too big to be represented by a normal JavaScript Number.

PROGRAM 6:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Bigint</h1>
<p>A BigInt can not have decimals.</p>
<p id="demo"></p>
<p>You cannot perform math between a BigInt type and a Number type.</p>
<script>
let x = BigInt("12345678901234567890123456789 0");
document.getElementById("demo").innerHTML = x;
</script>
</body>
</html>
```

Boolean

Booleans can only have two values: true or false.

PROGRAM 7:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Booleans</h2>
<p>Booleans can have two values: true or false:</p>
<p id="demo"></p>
<script>
let x = 5;
let y = 5;
let z = 6;
document.getElementById("demo").innerHTML =
(x == y) + "<br>" + (x == z);
</script>
</body>
</html>
```

Undefined

A variable without a value, has the value undefined. The type is also undefined.

PROGRAM 8:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Operators</h1>
<h2>The typeof Operator</h2>
<p>The value (and the data type) of a variable with no value is
<b>undefined</b>.</p>
<p id="demo"></p>
<script>
let car;
document.getElementById("demo").innerHTML =
car + "<br>" + typeof car;
</script>
</body>
</html>
```

Array

JavaScript arrays are written with square brackets, Array items are separated by commas.

PROGRAM 9:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Arrays</h2>
<p>Array indexes are zero-based, which means the first item is [0]. </p>
<p id="demo"></p>
<Script>
const cars = ["Saab","Volvo","BMW"];
document.getElementById("demo").innerHTML = cars[0];
</script>
</body>
</html>
```

Objects:

Objects are written with curly braces {}.Object properties are written as name: value pairs, separated by commas.

PROGRAM 10:

```
<! DOCTYPE html>
<html>
<Body>
<h2>JavaScript Objects</h2>
<p id="demo"></p>
<Script>
const person = {
  firstName : "John",
  lastName  : "Doe",
  age     : 50,
  eyeColor  : "blue"
};
document.getElementById("demo").innerHTML =
person.firstName + " is " + person.age + " years old.";
</script>
</body>
</html>
```

The type of Operator

You can use the JavaScript typeof operator to find the type of a JavaScript variable.The typeof operator returns the type of a variable or an expression:

PROGRAM 11:

```
<!DOCTYPE html>
<Html>
<Body>
<h1>JavaScript Operators</h1>
<h2>The typeof Operator</h2>
<p>the typeof operator returns the type of a variable or an expression. </p>
<p id="demo"></p>
<Script>
document.getElementById("demo").innerHTML =
typeof "" + "<br>" +
typeof "John" + "<br>" +
typeof 1045;
</script>
</body>
</html>
```

CHAPTER-3

3.1 JAVASCRIPT FUNCTIONS

A JavaScript function is defined with the function keyword, followed by a **name**, followed by parentheses **()**.Function names can contain letters, digits, underscores, and dollar signs (same rules as variables).The parentheses may include parameter names separated by commas:

Syntax:

```
function name(parameter1, parameter2, parameter3) {
  // code to be executed
}
```

(parameter1, parameter2,) code to be executed, by the function, is placed inside curly brackets: **{}**
Function **parameters** are listed inside the parentheses () in the function definition.
Function **arguments** are the **values** received by the function when it is invoked.Inside the function, the arguments (the parameters) behave as local variables.

Function Return Values:

JavaScript reaches a return statement, the function will stop executing. If the function was invoked from a statement, JavaScript will "return" to execute the code after the invoking statement. Functions often compute a **return value**. The return value is "returned" back to the "caller":

PROGRAM 1:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Functions</h1>
<p>Call a function which performs a calculation and returns the result:</p>
<p id="demo"></p>
<script>
let x = myFunction(4, 7);
document.getElementById("demo").innerHTML = x;
function myFunction(a, b) {
  return a * b;
}
</script>
</body>
</html>
```

3.2 FUNCTIONS USED AS VARIABLE VALUES

Functions can be used the same way as you use variables, in all types of formulas, assignments, and calculations.

PROGRAM 2:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Functions</h1>
<p>Using a function as a variable:</p>
<p id="demo"></p>
<script>
let text = "The temperature is " + toCelsius(77) + " Celsius.";
document.getElementById("demo").innerHTML = text;
function toCelsius(fahrenheit) {
  return (5/9) * (fahrenheit-32);
}
</script>
</body>
</html>
```

Local Variables

Variables declared within a JavaScript function, become **LOCAL** to the function. Local variables can only be accessed from within the function.

PROGRAM 3:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Functions</h1>
<p>Outside myFunction() carName is undefined.</p>
<p id="demo1"></p>
<p id="demo2"></p>
<script>
let text = "Outside: " + typeof carName;
document.getElementById("demo1").innerHTML = text;
function myFunction() {
  let carName = "Volvo";
  let text = "Inside: " + typeof carName + " " + carName;
  document.getElementById("demo2").innerHTML = text;
}
myFunction();
</script>
</body>
</html>
```

3.3 HTML EVENTS

An HTML event can be something the browser does, or something a user does. Here are some examples of HTML events:

- An HTML web page has finished loading
- An HTML input field was changed
- An HTML button was clicked

HTML allows event handler attributes, **with JavaScript code**, to be added to HTML elements.

With single quotes:

```
<element event='some JavaScript'>
```

With double quotes:

```
<element event="some JavaScript">
```

PROGRAM 4:

```
<!DOCTYPE html>
<html>
<body>
<button onclick="document.getElementById('demo').innerHTML=Date()">The time is?</button>
<p id="demo"></p>
</body>
</html>
```

Common HTML Events

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page

3.4 JAVASCRIPT STRINGS

JavaScript strings are for storing and manipulating text.

PROGRAM 5:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Strings</h1>
<p>You can use quotes inside a string, as long as they don't match the quotes
surrounding the string.</p>
<p id="demo"></p>
<script>
let answer1 = "It's alright";
let answer2 = "He is called 'Johnny'";
let answer3 = 'He is called "Johnny"';
document.getElementById("demo").innerHTML =
answer1 + "<br>" + answer2 + "<br>" + answer3;
</script>
</body>
</html>
```

3.5 JavaScript String Methods

String Length

To find the length of a string, use the built-in length property:

PROGRAM 6:

```
<!DOCTYPE html>
<html>
<body>
<p>The length of the string is:</p>
<p id="demo"></p>
<script>
let text = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
document.getElementById("demo").innerHTML = text.length;
</script>
</body>
</html>
```

Extracting String Parts

There are 3 methods for extracting a part of a string:

- slice(start, end)
- substring(start, end)
- substr(start, length)

Slice ()

Extracts a part of a string and returns the extracted part in a new string. The method takes 2 parameters: start position, and end position (end not included).

PROGRAM 7:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Strings</h1>
<h2>The slice() Method</h2>
<p>The sliced (extracted) part of the string is:</p>
<p id="demo"></p>
<script>
let text = "Apple, Banana, Kiwi";
let part = text.slice(7,13);
document.getElementById("demo").innerHTML = part;
</script>
</body>
</html>
```

Substring ()

Substring () is similar to slice ().The difference is that start and end values less than 0 are treated as 0 in substring ().

PROGRAM 8:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript String Methods</h1>
<p>The substring() method extract a part of a string and returns the extracted parts in a new string:</p>
<p id="demo"></p>
<script>
let str = "Apple, Banana, Kiwi";
```

```
document.getElementById("demo").innerHTML = str.substring(7,13);
</script>
</body>
</html>
```

Substr ()

Substr () is similar to slice ().The difference is that the second parameter specifies the **length** of the extracted part.

PROGRAM 9:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript String Methods</h1>
<p>The substr() method extract a part of a string
and returns the extracted parts in a new string:</p>
<p id="demo"></p>
<script>
let str = "Apple, Banana, Kiwi";
document.getElementById("demo").innerHTML = str.substr(7,6);
</script>
</body>
</html>
```

Replacing String Content

The replace () method replaces a specified value with another value in a string:

PROGRAM 10:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript String Methods</h1>
<p>Replace "Microsoft" with "W3Schools" in the paragraph below:</p>
<button onclick="myFunction()">Try it</button>
<p id="demo">Please visit Microsoft!</p>
<script>
function myFunction() {
  let text = document.getElementById("demo").innerHTML;
  document.getElementById("demo").innerHTML =
  text.replace("Microsoft","W3Schools");
  }
</script>
</body>
</html>
```

Converting to Upper and Lower Case

A string is converted to Uppercase () and to lowercase ():

PROGRAM 11:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript String Methods</h1>
<p>Convert string to upper case and lowercase:</p>
<button onclick="myFunction()">Try it</button>
<p id="demo">Hello World!</p>
<script>
function myFunction() {
  let text = document.getElementById("demo").innerHTML;
  document.getElementById("demo").innerHTML =
  text.toUpperCase();
  // tolowercase();
}
</script>
</body>
</html>
```

Concat ()

Concat () joins two or more strings:

PROGRAM 12:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript String</h1>
<h2>The concat() Method</h2>
<p>The concat() method joins two or more strings:</p>
<p id="demo"></p>
<script>
let text1 = "WELL";
let text2 = "COME!";
let text3 = text1.concat(" ",text2);
document.getElementById("demo").innerHTML = text3;
</script>
</body>
</html>
```

String Split ()
A string can be converted to an array with the split () method:
PROGRAM 13:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript String Methods</h1>
<p>Display the first array element, after a string split:</p>
<p id="demo"></p>
<script>
let text = "a,b,c,d,e,f";
const myArray = text.split(",");
document.getElementById("demo").innerHTML = myArray[0];
</script>
</body>
</html>
```

3.6 STRING SEARCH METHODS

- String indexOf()
- String lastIndexOf()
- String matchAll()
- String includes()
- String startsWith()
- String endsWith()

Indexof ()
The indexOf () method returns the index (position) the first occurrence of a string in a string:
PROGRAM 14:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Strings</h1>
<h2>The indexOf() Method</h2>
<p>The indexOf() method returns the position of the first occurrence of a string in a
string.</p>
<p>The position of the first occurrence of "locate" is:</p>
<p id="demo"></p>
<script>
let text = "Please locate where 'locate' occurs!";
let index = text.indexOf("locate");
document.getElementById("demo").innerHTML = index;
</script>
</body>
</html>
```

LastIndexOf ()

The lastIndexOf () method returns the **index** of the **last** occurrence of a specified text in a string:

PROGRAM 15:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Strings</h1>
<h2>The indexOf() Method</h2>
<p>Both indexOf() and lastIndexOf() return -1 if the text is not found:</p>
<p id="demo"></p>
<script>
let text = "Please locate where 'locate' occurs!";
let index = text.indexOf("wellcome");
document.getElementById("demo").innerHTML = index;
</script>
</body>
</html>
```

String match All ()

The matchAll () method returns an iterator containing the results of matching a string against a string (or a regular expression).

PROGRAM 16:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Strings</h1>
<h2>The matchAll() Method</h2>
<p>ES2020 intoduced the string method matchAll().</p>
<p id="demo"></p>
<script>
let text = "I love cats. Cats are very easy to love. Cats are very popular."
const iterator = text.matchAll("Cats");
document.getElementById("demo").innerHTML = Array.from(iterator);
</script>
</body>
</html>
```

String includes ()

The includes () method returns true if a string contains a specified value. Otherwise it returns false.

PROGRAM 17:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Strings</h1>
<h2>The includes() Method</h2>
<p>Check if a string includes "world":</p>
<p id="demo"></p>
<p>The includes() method is not supported in Internet Explorer.</p>
<script>
let text = "Hello world, welcome to the universe.";
document.getElementById("demo").innerHTML = text.includes("world");
</script>
</body>
</html>
```

String startsWith () and endsWith ()

The JavaScript starts with () method is used to determine whether a string starts with a particular substring. The ends with () method is used for the same purpose but checks whether a string ends with a specified substring.

PROGRAM 18:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Strings</h2>
<p>Check if a string starts and ends with "Doe":</p>
<p id="demo"></p>
<p>The starts and endsWith() method is not supported in Internet Explorer.</p>
<script>
let text = "John Doe";
document.getElementById("demo").innerHTML = text.endsWith("Doe");
//text.startWith("John");
</script>
</body>
</html>
```